Phone Calls Through Vapi

How the AI Receptionist is configured, how a call flows through it, and the issues we hit while connecting it

Prepared 23 October 2026 | Branch feat/platform-hours-and-confirmations

1. Summary

A caller dials a business's phone number. **Vapi** answers the line, listens, and speaks. Our backend decides *what* is said: it finds the business from the number that was dialled, then runs that business's prompt, language model and booking tools. A real test call to **Sunrise Grand Hotel** now works end to end: the number reached our webhook, then the per-turn endpoint received the caller's speech, and Vapi's call log shows the business's assistant.

- **Vapi does:** the phone line, speech-to-text, text-to-speech, and interruptions (barge-in).
- **Our backend does:** tenant lookup from the dialled number, the business prompt and hours, the LLM, the business's own MCP tools, saving conversations and usage, and alerting staff when a caller needs a person.
- **Number:** a free Vapi US number (+1 769 263 0020) is saved on the Phone page against Sunrise Grand Hotel.

2. How a call flows

```
Caller phone  --->  Vapi number (+1 769 263 0020)
                      |
                      |  1. assistant-request  (POST /api/vapi/webhook)
                      v
                 Our backend: finds the business from the dialled number,
                 starts a conversation, returns that business's assistant
                      |
                      |  2. every turn: caller's words (POST /api/vapi/llm/{token}/...)
                      v
                 ConversationService -> LLM + the business's MCP tools
                      |
                      |  3. reply streamed back as text (server-sent events)
                      v
                 Vapi speaks it to the caller (Deepgram voice)

End of call: Vapi sends status-update and end-of-call-report to the same webhook.
```

Step by step

Step	What happens	Where in the code
1. Call arrives	Vapi sends an **assistant-request** to our webhook with the secret in the X-Vapi-Secret header. We reject a missing or wrong secret (403).	api/vapi_routes.py: webhook()
2. Find the business	The dialled number is read from **message.phoneNumber.number**, converted to E.164, and looked up in the phone_numbers table. Unknown number: Vapi is told it is not in service.	voice/vapi.py: call_numbers(); repositories/phone.py: business_for_number()
3. Start the call	A conversation is created with the greeting; channel, call id and the caller's number are saved in its context. A signed per-call token (audience vapi-llm, 2 hours) names the business and conversation.	ConversationService.start(); vapi.create_llm_token()
4. Return the assistant	We reply with the greeting, language, a Deepgram transcriber and voice, and a custom-llm URL on this server carrying the token.	vapi.assistant_config()

Step	What happens	Where in the code
5. Each turn	Vapi posts the transcript to the custom-llm URL. We take the newest user message, run the business's prompt, LLM and tools, and stream the answer back as chat-completion chunks.	api/vapi_routes.py: chat_completions()
6. Problems	If the assistant fails, the caller is told, offered a person, and on yes the business's staff are texted and emailed once.	ConversationService._turn(); notifications/staff.py
7. End of call	status-update and end-of-call-report are acknowledged with an empty reply. They are not stored yet.	vapi_routes.webhook()

When something goes wrong during a call

- **The assistant or LLM fails:** the caller hears that we are having trouble with our system and is offered a person (or a callback if the business is closed or has no transfer number).
- **The caller says yes:** handled in plain code, not by the LLM, because the LLM may still be down. The conversation is flagged and the business is pinged once: an SMS to the transfer number and an email to the business's active admins.
- **A business tool (for example booking) fails:** the prompt tells the assistant to name the part of the system that is having trouble and offer a person; on yes it calls request_human, which triggers the same ping.
- **A failed ping** is logged and never breaks the call. Yes/no detection is English only.

3. Configuration

3.1 Backend settings (.env)

Setting	Purpose	Notes
PUBLIC_BASE_URL	Public https URL of the backend. Used to build the webhook and per-call LLM URLs.	Currently an ngrok URL. It changes if ngrok restarts on a free plan without a fixed domain.
VAPI_WEBHOOK_SECRET	Shared secret Vapi sends with every server message. Empty means Vapi calls are refused.	Use a long random value. Never commit it.
VAPI_LLM_TOKEN_SECONDS	How long a call's LLM URL stays valid (default 7200).	Optional; the default is longer than any call.
LLM_PROVIDER and key	The model that writes replies (for example GROQ_API_KEY).	Existing setting.
DEEPGRAM_API_KEY	Used by the browser voice path and the Twilio path. Vapi uses its own Deepgram voice.	Existing setting.
SENDGRID_API_KEY, EMAIL_FROM, Twilio SMS values	Make staff alerts and confirmations actually send.	Empty in development: messages are only written to the log.

After editing .env, recreate the container so it is re-read: `docker compose up -d backend`. A plain restart may not load new values.

3.2 Vapi dashboard

- **Phone Numbers - Create - Free Vapi Number.** Free US numbers only, one per subscription on this plan, inbound calls only.
- **Server URL** on the number: `<PUBLIC_BASE_URL>/api/vapi/webhook`. Timeout can stay at 20 seconds (Vapi itself requires an answer to an assistant-request within 7.5 seconds).
- **HTTP Header** on the number: name `X-Vapi-Secret`, value the same as VAPI_WEBHOOK_SECRET. (A Bearer credential would also work; the header was simpler.)
- **Assistant** field: leave empty, so Vapi asks our server for an assistant on every call.
- The Test button in Vapi tries an outbound call and will always fail on a free number. Ignore it.

3.3 Our app

- Business admin opens **Phone**, enters the Vapi number in E.164 form (+17692630020) and an optional transfer number, then saves. One number per business; numbers are unique across businesses.
- The page now shows the right connection instructions: when VAPI_WEBHOOK_SECRET is set it shows the Vapi server URL, otherwise the Twilio webhook URL.
- ngrok forwards the public URL to `localhost:8000`. Its inspector at `127.0.0.1:4040` shows every request and body, which is how the last bug was found.

4. Issues we faced and how we fixed them

In the order we met them. Where a cause was not confirmed, the table says so.

#	Issue	Cause	Fix / outcome
1	Twilio account SID and auth token were saved in a local git stash of .env.example and later shown in screenshots.	Real credentials were pasted into an example file; later screenshots also exposed them. Nothing was found on the remote branches.	Working copy of .env.example is empty of secrets. **Still to do:** rotate the Twilio auth token and the Vapi secret header, since both appeared in screenshots.
2	Calling the Twilio number rang and completed, but Vapi Logs were empty and the backend log showed no call requests.	The number was still answered by Twilio's own default setup, not by Vapi or us.	Number connected to Vapi instead (issues 3 and 4). Confirmed afterwards by requests appearing in ngrok and the backend.
3	Vapi import of the Twilio number failed: 'Number Not Found on Twilio'.	Not confirmed. Likely a stale or wrong auth token, or limits of the Twilio trial account.	Skipped. Used a free Vapi number instead, which needs no Twilio account.
4	Twilio asked to upgrade the plan.	Trial accounts keep one trial number; buying more needs an upgrade.	Not needed for testing. Upgrade only for real customers (more numbers, texting unverified numbers).
5	Free Vapi numbers are US only; the tester is in India.	Vapi's free numbers support US area codes only.	Called the US number as an international call. Real Indian customers will need an Indian number brought into Vapi (cloud-telephony SIP trunk, or an imported Twilio/Telnyx number) - a product and cost decision.
6	Phone page still said 'Connect it in Twilio' and showed /api/phone/incoming.	The page only knew the Twilio path.	Backend now returns a provider and URL; the page shows 'Connect it in Vapi' with /api/vapi/webhook when VAPI_WEBHOOK_SECRET is set. Tests added.
7	Saving the number on the Phone page returned 401 Unauthorized.	The admin login had expired; the save was rejected and was not retried.	Logged in again and saved. Verified in the database that the number belongs to the right business.
8	**Every call said 'this number is not in service'**, although the number was saved and the webhook returned 200.	**Bug in our code.** For an assistant-request, Vapi sends the dialled number in message.phoneNumber.number and leaves call.phoneNumber empty (null). We only read call.phoneNumber, so the number was never found.	Found by reading the real request bodies through ngrok's local API. call_numbers() now reads the message first and uses the call as a fallback. Tests rewritten to use Vapi's real shape; 46 tests pass. A real call then worked.
9	Vapi Logs showed Outbound rows with Call.start.error.	They came from the dashboard's Test button; Vapi numbers on this plan cannot make outbound calls.	Not an application problem. Ignore the Test button.
10	Long pydantic 'alias has no effect' warnings in the backend log.	Raised while building a schema; no such alias exists in our code, so it comes from a library.	Harmless; no action taken.
11	Config changes did not take effect in Docker.	A plain container restart may not re-read .env.	Use docker compose up -d backend to recreate the container.

5. What was verified

Check	Result
Automated tests: Vapi, phone and staff-alert files	46 passed (after the fix).
Full backend suite (earlier run)	202 passed, 2 failed. Both failures are in test_engine.py and are caused by the openai package not being installed in the local Python, not by this work.
Phone page tests (frontend)	4 passed, including the Vapi variant.
Real call to the Vapi number	Reached /api/vapi/webhook, then repeated requests to /api/vapi/llm/... for each caller turn (ngrok), and Vapi's Logs show the Sunrise Grand Hotel assistant on an inbound call.

6. Known gaps and next steps

- **Rotate secrets:** Twilio auth token and the X-Vapi-Secret value were visible in screenshots.
- **Ended Reason shows 'Unknown'** on the successful call in Vapi's log. We have not investigated it; it may simply be how Vapi labels a custom-LLM call.
- **End of call is not stored:** no duration, no voice-minute usage, conversations are not marked ended.
- **Live transfer to a person** on Vapi is not built (the caller is told, and staff are pinged).
- **One-click number provisioning** from the Phone page is not built; numbers are added in the Vapi dashboard by hand.
- **Languages:** the language is fixed per business; it does not follow what the caller speaks.
- **Real customers need real numbers:** free Vapi numbers are US-only and limited; plan paid or Indian numbers, and keep one number per business.
- **ngrok:** for production the backend needs a permanent public URL.
- The earlier Twilio media-stream path is still in the code as a fallback.